7

ПРОЕКТИРОВАНИЕ ГЕНЕРАТОРА УНИКАЛЬНЫХ ИДЕНТИФИКАТОРОВ В РАСПРЕДЕЛЕННЫХ СИСТЕМАХ

В этой главе вам предлагается спроектировать генератор уникальных идентификаторов в распределенной системе. Возможно, использование первичного ключа с атрибутом `auto_increment` в традиционной базе данных станет первым, что придет вам в голову. Однако этот подход не работает в распределенных окружениях, так как отдельно взятый сервер БД слишком мал. К тому же генерировать уникальные ID в рамках нескольких БД с минимальными задержками не так уж просто.

Вот несколько примеров уникальных идентификаторов:

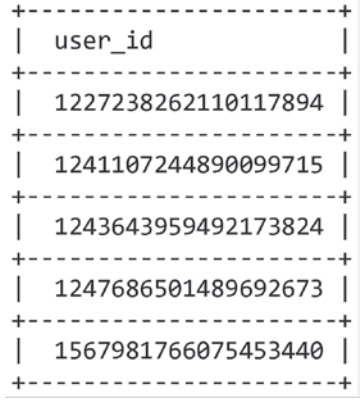


Рис. 7.1

ШАГ 1: ПОНЯТЬ ЗАДАЧУ И ОПРЕДЕЛИТЬ МАСШТАБ РЕШЕНИЯ

Постановка уточняющих вопросов — это первый шаг к решению задачи на любом интервью по проектированию ИТ-систем. Вот пример диалога между кандидатом и интервьюером:

Кандидат: «Какие характеристики должны быть у уникальных ID?»

Интервьюер: «ID должны быть уникальными и подлежать сортировке».

Кандидат: «Инкрементируется ли ID на 1 при добавлении каждой новой записи?»

Интервьюер: «ID инкрементируется по времени, но необязательно на 1. Идентификаторы, созданные вечером, больше тех, которые были получены утром того же дня».

Кандидат: «ID имеют только числовые значения?»

Интервьюер: «Да, именно так».

Кандидат: «Какие требования к длине ID?»

Интервьюер: «ID должны умещаться в 64 бита».

Кандидат: «Какой масштаб системы?»

Интервьюер: «Система должна быть способна генерировать 10 000 ID в секунду».

Это лишь некоторые из вопросов, которые можно задать интервьюеру. Необходимо разобраться в требованиях и прояснить непонятные моменты. В этом примере к системе предъявляются следующие требования:

- ID должны быть уникальными;
- ID должны быть сугубо числовыми;
- ID должны умещаться в 64 бита;
- ID должны быть упорядочены по дате;
- система должна генерировать 10 000 ID в секунду.

ШАГ 2: ПРЕДЛОЖИТЬ ОБЩЕЕ РЕШЕНИЕ И ПОЛУЧИТЬ СОГЛАСИЕ

Есть разные методы генерации уникальных ID в распределенных системах. Мы рассмотрели следующие варианты:

- репликация с несколькими источниками;
- универсальный уникальный идентификатор (universally unique identifier, UUID);
- сервер тикетов;
- Twitter snowflake ID — подход «снежного кома» Twitter.

Давайте обсудим каждый из них. Рассмотрим принципы их работы, а также их слабые и сильные стороны.

Репликация с несколькими источниками

На рис. 7.2 показан первый подход — репликация с несколькими источниками.

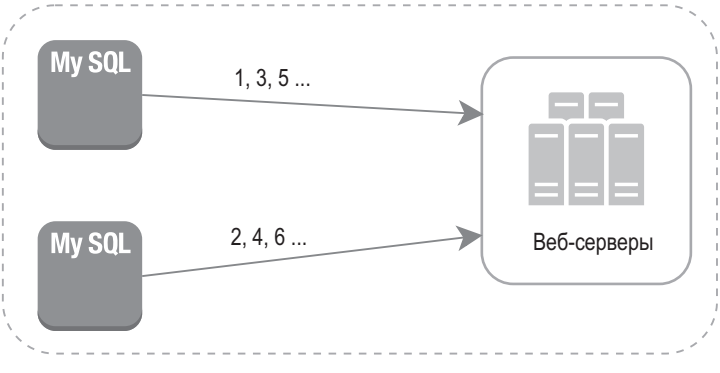


Рис. 7.2

Здесь используется свойство баз данных `auto_increment`. ID увеличивается не на 1, а на число *k*, равное количеству используемых БД. Как проиллюстрировано на рис. 7.2, следующий ID равен предыдущему, сгенерированному на том же сервере, плюс 2. Это позволяет избежать определенных проблем с масштабированием, так как идентификаторы

могут увеличиваться вместе с количеством серверов БД. Однако у этой стратегии есть серьезные недостатки:

- сложность масштабирования в конфигурации с несколькими центрами обработки данных;
- ID не увеличиваются хронологически в пределах нескольких серверов;
- плохая масштабируемость при добавлении или удалении сервера.

UUID

Это еще один простой способ получения уникальных идентификаторов. UUID — это 128-битное число, которое используется для идентификации данных в компьютерных системах. UUID имеют очень низкую вероятность повторения. Цитата из Википедии: «После генерации 1 миллиарда UUID в секунду на протяжении примерно 100 лет вероятность получения одного дубликата достигает 50 %» [1].

UUID может выглядеть как 09c93e62-50b4-468d-bf8a-c07e1040bfb2. Эти идентификаторы можно генерировать независимо друг от друга без координации между серверами. Архитектура UUID представлена на рис. 7.3.

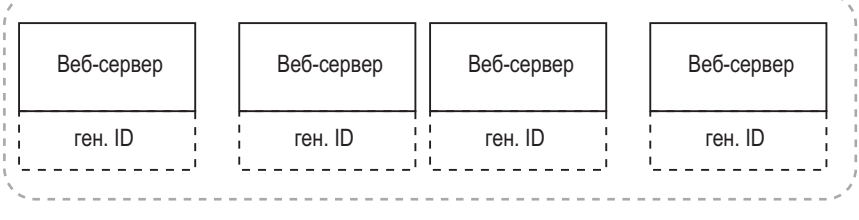


Рис. 7.3

В этой конфигурации каждый веб-сервер содержит генератор идентификаторов, работающий независимо от остальных серверов.

Преимущества:

- простота генерации. Не нужно никакой координации между серверами, что исключает проблемы с синхронизацией;

- систему легко масштабировать, так как каждый сервер отвечает за генерацию идентификаторов, которые он потребляет. Генератор может легко масштабироваться вместе с веб-серверами.

Недостатки:

- ID имеют длину 128 бит, а нам нужно 64 бита;
- ID не увеличиваются со временем;
- ID могут быть нечисловыми.

Сервер тикетов

Серверы тикетов[1] — это еще один интересный способ генерации уникальных ID. Их разработала компания Flicker для получения распределенных первичных ключей [2]. Давайте посмотрим, как они работают.

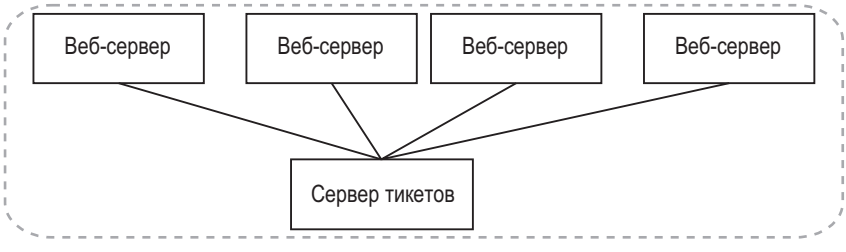


Рис. 7.4

Суть в том, что мы используем функцию `auto_increment` в отдельно взятом сервере баз данных (сервере тикетов). Подробнее об этом можно почитать в статье блога разработчиков Flicker [2].

Преимущества:

- числовые ID;
- этот метод легко реализовать, и он подходит для небольших и средних приложений.

[1] В русскоязычных описаниях также встречается вариант «сервер билетов». — *Примеч. ред.*

Недостатки:

- единая точка отказа. Сервер тикетов существует в единственном экземпляре, и если он выйдет из строя, проблемы возникнут у всех систем, которые на него полагаются. Чтобы этого избежать, можно предусмотреть несколько серверов тикетов, но это вызовет новые трудности, такие как синхронизация данных.

Twitter snowflake ID

Методики, упомянутые выше, позволяют получить некоторое представление о работе разных систем генерации ID. Но ни одна из них не отвечает нашим требованиям, поэтому нам нужен другой подход. Интересным вариантом, способным удовлетворить наши нужды, является система генерации уникальных ID от Twitter под названием snowflake.

Разделяй и властвуй — вот что нам нужно. Вместо того чтобы генерировать идентификатор напрямую, мы разделяем его на части. На рис. 7.5 показана структура 64-битного ID.

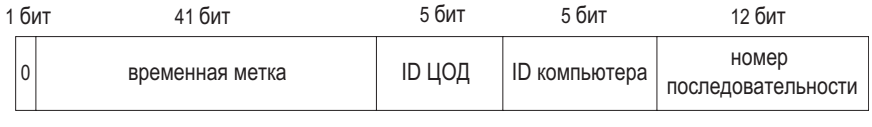


Рис. 7.5

Каждая часть описана ниже.

- Бит знака: 1 бит. Всегда равен 0 и зарезервирован на будущее. С его помощью потенциально можно различать знаковые и беззнаковые числа.
- Временная метка: 41 бит. Количество миллисекунд, прошедших с начала эпохи Unix или какого-то другого момента. В Twitter snowflake начальной точкой по умолчанию является Ноя 04, 2010, 01:42:54 UTC, что эквивалентно 1288834974657. Мы воспользуемся этим значением.
- ID ЦОД: 5 бит, что дает нам 2 ^ 5 = 32 центра обработки данных.
- ID компьютера: 5 бит, что дает нам 2 ^ 5 = 32 компьютера в каждом ЦОД.

- Номер последовательности: 12 бит. При генерации каждого ID на отдельно взятом компьютере или процессе номер последовательности инкрементируется на 1. Каждую миллисекунду этот номер обнуляется.

ШАГ 3: ПОДРОБНОЕ ПРОЕКТИРОВАНИЕ

При обсуждении общих аспектов проектирования мы перечислили разные способы генерации уникальных ID в распределенных системах и остановились на подходе, основанном на генераторе snowflake ID от Twitter. Давайте подробно рассмотрим эту архитектуру. Чтобы освежить память, еще раз приведем диаграмму snowflake ID.



Рис. 7.6

ID центра обработки данных и компьютера выбираются в момент запуска системы и обычно не меняются во время выполнения. Любые обновления этих идентификаторов требуют тщательного анализа, так как неосторожное внесение изменений может привести к конфликтам. Временные метки и номера последовательностей генерируются в ходе работы системы.

Временная метка

Самую важную часть идентификатора составляет 41-битная временная метка. Поскольку метки увеличиваются со временем, ID можно сортировать в хронологическом порядке. На рис. 7.7 показан пример преобразования двоичного представления в UTC. Обратное преобразование можно выполнить аналогичным образом.

Максимальная временная метка, которую можно представить с помощью 41 бита, равна 2 ^ 41 - 1 = 2199023255551 миллисекундам (мс), что дает нам ~69 лет = 2199023255551 мс / 1000 / 365 дней / 24 часов / 3600 секунд. Это означает, что этот генератор ID будет работать на протяжении 69 лет, и чем ближе начало эпохи к текущей дате, тем позже

произойдет переполнение. По прошествии этого времени нужно будет установить новую эпоху или внедрить другой метод для миграции идентификаторов.

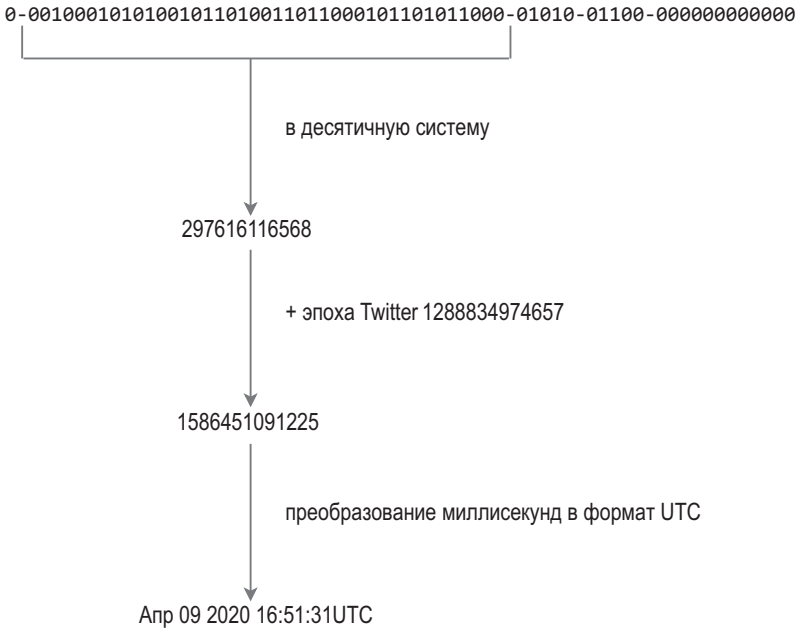


Рис. 7.7

Номер последовательности

Номер последовательности занимает 12 бит, что дает нам 2 ^ 12 = 4096 комбинаций. Это поле не равно нулю только в случае, если на одном и том же сервере с одну миллисекунду сгенерировано больше одного ID. Теоретически компьютер может генерировать до 4096 новых ID в миллисекунду.

ШАГ 4: ПОДВЕДЕНИЕ ИТОГОВ

В этой главе мы обсудили разные подходы к проектированию генератора уникальных идентификаторов: репликация с несколькими источниками, UUID, сервер тикетов и генератор вида Twitter snowflake. Мы останови-

лись на последнем варианте, так как он отвечает всем нашим требованиям и способен масштабироваться в распределенном окружении.

Если в конце интервью остается время, можно обсудить дополнительные вопросы.

- Синхронизация часов. В нашей архитектуре предполагается, что у серверов, генерирующих ID, часы синхронизированы. Это может быть не так, если мы используем многоядерный компьютер или конфигурацию с несколькими серверами. В этой книге способы синхронизации часов не рассматриваются; просто знайте, что такая проблема существует. Самым популярным ее решением является протокол NTP (Network Time Protocol — «протокол сетевого времени»). Если вас это заинтересовало, можете обратиться к справочному материалу [4].
- Оптимизация длины отдельных частей. Например, выделение большего числа битов для временной метки за счет уменьшения номера последовательности хорошо подходит для приложений, которые долго работают и имеют низкую степень параллелизма.
- Высокая доступность. Поскольку генератор ID является незаменимой системой, он должен быть высокодоступным.

Поздравляем, вы проделали длинный путь и можете гордиться собой. Отличная работа!

СПРАВОЧНЫЕ МАТЕРИАЛЫ

[1] UUID: https://ru.wikipedia.org/wiki/UUID

[2] Ticket Servers: Distributed Unique Primary Keys on the Cheap: https://code.flickr.net/2010/02/08/ticket-servers-distributed-unique-primary-keys-on-the-cheap/

[3] Announcing Snowflake: https://blog.twitter.com/engineering/en_us/a/2010/announcing-snowflake.html

[4] Протокол NTP: https://ru.wikipedia.org/wiki/NTP

8

ПРОЕКТИРОВАНИЕ СИСТЕМЫ ДЛЯ СОКРАЩЕНИЯ URL-АДРЕСОВ

В этой главе мы рассмотрим интересную классическую задачу, с которой можно столкнуться на интервью по проектированию ИТ-систем: разработка сервиса для сокращения URL-адресов по примеру tinyurl.

ШАГ 1: ПОНЯТЬ ЗАДАЧУ И ОПРЕДЕЛИТЬ МАСШТАБ РЕШЕНИЯ

На интервью по проектированию ИТ-систем специально дают задачи, допускающие свободную трактовку. Чтобы разработать хорошо продуманную систему, необходимо задавать уточняющие вопросы.

Кандидат: «Можете ли привести пример работы сервиса для сокращения URL-адресов?»

Интервьюер: «Допустим, https://www.systeminterview.com/q=chatsystem&c=loggedin&v=v3&l=long — это исходный URL-адрес. Ваш сервис должен создать ссылку покороче: https://tinyurl.com/y7keocwj. Клик по этой ссылке должен перенаправлять к исходному URL-адресу».

Кандидат: «Какой объем трафика?»

Интервьюер: «100 миллионов сгенерированных URL-адресов в день».

Кандидат: «Какая длина должна быть у сокращенного URL-адреса?»

Интервьюер: «Как можно короче».

Кандидат: «Какие символы допускаются в сокращенном URL-адресе?»

Интервьюер: «Сокращенный URL-адрес может содержать цифры (0–9) и буквы (a–z, A–Z)».